

Terminal & Git Commands — Cheat Sheet

A quick reference for moving around the command line, managing files, and working with Git & GitHub.

1. Navigation & File Management

These commands let you move through project folders and manage files.

Command	Purpose	Example / Details
pwd	Print Working Directory. Shows the full path of your current folder.	—
ls	Lists all files and folders in the current directory.	Use ls -la to see hidden files (e.g. .env , .gitignore).
cd [folder]	Change Directory. Moves you into a specific folder.	Use cd .. to go up one level.
mkdir [name]	Make Directory. Creates a new directory.	—
touch [filename]	Creates a new, empty file.	e.g. touch index.html
cp / mv	Copy or Move/rename files and folders.	—
rm [file]	Remove. Deletes a file.	Use rm -rf [folder] to permanently delete a folder and its contents. No Trash bin — use with caution.

2. Development Utilities

Inspect code and troubleshoot directly from the command line.

Command	Purpose	Example / Details
cat [file]	Displays the entire contents of a file in the terminal.	—
grep "[term]" [file]	Searches for a specific string of text within a file or directory.	—
curl [url]	Sends a request to a URL.	Web devs use this to test APIs or download files.
chmod	Changes file permissions.	Often used to make a script executable.
history	Shows a list of all commands you have recently typed.	—

3. Essential Keyboard Shortcuts

Built-in terminal shortcuts that save real time.

Shortcut	Purpose
Tab	Auto-completes the name of a file or folder.
Up / Down Arrows	Cycles through your previous commands.
Ctrl + C	Immediately stops a running process (like a local dev server).
Ctrl + L / clear	Clears the terminal screen.

4. Web Dev Ecosystem Commands

Not built into the terminal, but you'll run these daily.

Command	Purpose	Example / Details
git	Version control system.	git status, git add ., git commit
npm / yarn	Package managers for Node.js.	Install packages (npm install) and run scripts (npm start).
ssh	Secure Shell. Securely connects you to a remote server.	Used to deploy or manage your site remotely.

5. GitHub Setup (one-time)

Do these once on each machine you work from. After this you can push, pull, and clone freely.

Step	What to do
1. Create an account	Sign up at github.com . Use a professional username — you'll be sharing this link on your résumé.
2. Install Git	macOS: brew install git • Windows: download from git-scm.com • Linux: sudo apt install git . Check it worked: git --version .
3. Set your identity	Tells Git who you are. Run these once per machine: git config --global user.name "Your Name" git config --global user.email "you@example.com"
4. Set the default branch	Modern repos use main instead of master : git config --global init.defaultBranch main
5. Authenticate with GitHub	Two options — pick one: • HTTPS + Personal Access Token (easiest) : GitHub → Settings → Developer settings → Personal access tokens → generate one with <i>repo</i> scope. Paste it as the password when Git prompts. • SSH (more setup, smoother long-term) : see step 6.
6. (Optional) SSH key setup	Generate a key: ssh-keygen -t ed25519 -C "you@example.com" Copy the public key: cat ~/.ssh/id_ed25519.pub Paste it into GitHub → Settings → SSH and GPG keys → New SSH key. Test it: ssh -T git@github.com
7. Verify	Make a test repo on GitHub, clone it, push a change. If that works, you're set.

6. Essential Git Commands

The day-to-day commands. Memorise **status**, **add**, **commit**, **push**, **pull** first — the rest follow.

Command	Purpose	Example / Details
git init	Turns the current folder into a Git repository.	Run this once at the root of a new project.
git clone [url]	Downloads a remote repo to your machine.	git clone https://github.com/user/repo.git
git status	Shows changed, staged, and untracked files.	Use this constantly. It's your map.
git add [file]	Stages a file for the next commit.	git add . stages everything.
git commit -m "msg"	Saves staged changes as a snapshot with a message.	Keep messages short and present-tense: "add login form"
git log	Shows commit history for the current branch.	Press q to exit. Try git log --oneline for a tidy view.

Command	Purpose	Example / Details
git diff	Shows unstaged changes (what you'd add if you ran add).	git diff --staged shows what you'd commit.
git branch	Lists branches, or creates a new one.	git branch feature-x creates one named feature-x.
git checkout [branch]	Switches to an existing branch.	git checkout -b feature-x creates + switches in one step.
git switch [branch]	Modern replacement for checkout (switching only).	git switch -c feature-x creates and switches.
git merge [branch]	Merges another branch into your current branch.	On main , run git merge feature-x to pull it in.
git pull	Fetches and merges remote changes into your current branch.	Do this before you start work each day.
git push	Uploads your local commits to GitHub.	First push of a new branch: git push -u origin [branch]
git remote -v	Shows the remote URLs Git is connected to.	Handy when something says "remote not found".
git remote add origin [url]	Links your local repo to a GitHub repo.	Used once after git init .
.gitignore	A file listing things Git should ignore (e.g. node_modules , .env).	Create at project root before your first commit.

7. Common Git Workflows

Copy-paste recipes for the situations you'll hit weekly.

Scenario	Commands (in order)
Start a new local project and push it to GitHub	<pre>git init git add . git commit -m "initial commit" git branch -M main git remote add origin [your-repo-url] git push -u origin main</pre>
Save your work for the day	<pre>git status git add . git commit -m "describe what changed" git push</pre>
Get the latest changes from GitHub	<pre>git pull</pre>
Work on a new feature safely on a branch	<pre>git switch -c feature-x # do your work, commit as you go git push -u origin feature-x # open a Pull Request on GitHub</pre>

Scenario	Commands (in order)
Undo changes to a file you haven't committed yet	<code>git restore [file]</code>
Unstage a file (keep the changes, just don't commit them)	<code>git restore --staged [file]</code>
See what you committed in the last hour of panic	<code>git log --oneline -10</code>